

Phase 5

Project Development Phase

Setup, configuration and core backend implementation

1. Development Environment Setup

Create a folder named **AI StudyBuddy**, open it in Visual Studio Code and run the following in the terminal.

```
npm init -y

npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai

npm install --save-dev nodemon
```

Required files: `index.js`, `.env`, `package.json`, `.gitignore`.

Environment variables (.env)

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
GEMINI_API_KEY=your_gemini_api_key
```

Check before running: (1) the auth middleware in the source reads `JWT_ACCESS_SECRET`, so define that name in `.env` (or change the code to match `JWT_SECRET`); (2) `index.js` and `gemini.js` import `express-async-errors` and `@google/generative-ai`, which are not in the install command above, so install them as well.

2. Backend Structure

Routing, controllers, middleware, models and utility modules are separated for maintainability and scalability.

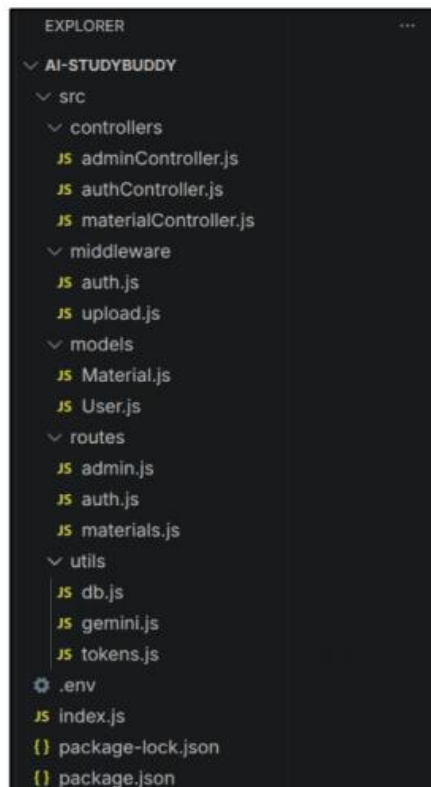


Figure: Backend folder structure

3. Core Implementations

3.1 Application entry point (index.js)

Initializes Express, loads environment variables, configures middleware, connects MongoDB, registers routes and starts the server after a successful database connection.

```
require("dotenv").config();
require("express-async-errors");

const express = require("express");
const cors = require("cors");
const connectDB = require("../src/utils/db");

const app = express();

// Middleware
app.use(express.json());
app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));

// Routes
app.use("/api/auth", require("../src/routes/auth"));
app.use("/api/materials", require("../src/routes/materials"));
app.use("/api/admin", require("../src/routes/admin"));

// Health check
app.get("/", (req, res) => res.json({ message: "AI StudyBuddy API is running" }));

// Global error handler
app.use((err, req, res, next) => {
  console.error(err.message);
  const status = err.status || 500;
  res.status(status).json({ message: err.message || "Something went wrong" });
});

const PORT = process.env.PORT || 5000;

connectDB().then(() => {
  app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
});
```

3.2 Database connection (src/utils/db.js)

Connects the backend to MongoDB using Mongoose before any client request is processed.

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");

dotenv.config();

const connectDB = async () => {
  const conn = await mongoose.connect(process.env.MONGO_URI);
  console.log(`MongoDB connected: ${conn.connection.host}`);
};

module.exports = connectDB;
```

3.3 Authentication middleware (src/middleware/auth.js)

Verifies JWT access tokens for protected routes and restricts administrative operations by role.

```
const jwt = require("jsonwebtoken");

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ message: "Not authenticated" });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
    req.user = decoded;
    next();
  } catch {
    return res.status(401).json({ message: "Invalid or expired token" });
  }
};

const adminOnly = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({ message: "Admins only" });
  }
  next();
};

module.exports = { protect, adminOnly };
```

3.4 AI integration (src/utils/gemini.js)

Sends prompts to the Google Gemini 2.5 Flash model and returns generated text for summaries, flashcards, quizzes and study plans.

```
const { GoogleGenerativeAI } = require("@google/generative-ai");

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash" });

const askGemini = async (prompt) => {
  const result = await model.generateContent(prompt);
  return result.response.text();
};

module.exports = { askGemini };
```

3.5 File upload middleware (src/middleware/upload.js)

Handles study-material uploads and validates incoming files (.txt, .md, .pdf; maximum 5 MB).

```
const multer = require("multer");
const path = require("path");

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, "uploads/"),
  filename: (req, file, cb) => {
```

```

        const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
        cb(null, unique + path.extname(file.originalname));
    },
    });

const fileFilter = (req, file, cb) => {
    const allowed = [".txt", ".md", ".pdf"];
    const ext = path.extname(file.originalname).toLowerCase();
    if (allowed.includes(ext)) {
        cb(null, true);
    } else {
        cb(new Error("Only .txt, .md, and .pdf files are allowed"));
    }
};

const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });

module.exports = upload;

```

4. Database Models (Mongoose)

4.1 User schema (src/models/User.js)

Stores authentication and profile information. Passwords are hashed with bcrypt before saving.

```

const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema(
    {
        name: { type: String, required: true, trim: true },
        email: { type: String, required: true, unique: true, lowercase: true },
        password: { type: String, required: true },
        role: { type: String, enum: ["student", "admin"], default: "student" },
    },
    { timestamps: true }
);

// Hash password before saving
userSchema.pre("save", async function (next) {
    if (!this.isModified("password")) return next();
    this.password = await bcrypt.hash(this.password, 10);
    next();
});

userSchema.methods.matchPassword = function (password) {
    return bcrypt.compare(password, this.password);
};

module.exports = mongoose.model("User", userSchema);

```

4.2 Study Material schema (src/models/Material.js)

Stores uploaded learning content and the AI-generated summary, flashcards, quiz and study plan for that material.

```

const mongoose = require("mongoose");

const materialSchema = new mongoose.Schema(
    {
        user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
        title: { type: String, required: true },
        content: { type: String, required: true }, // raw text content
        filename: { type: String }, // original uploaded file name
        summary: { type: String },
        flashcards: [
            {
                question: String,
                answer: String,
            },
        ],
        quiz: [
            {
                question: String,
            },
        ],
    },
    { timestamps: true }
);

module.exports = mongoose.model("Material", materialSchema);

```

```
        options: [String],
        answer: String,
      },
    ],
    studyPlan: { type: String },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Material", materialSchema);
```

Database collections: *Users* (account and authentication details) and *Study Materials* (uploaded content with generated AI resources).

5. API Endpoints Implemented

Category	Description
Authentication APIs	User registration, login, JWT authentication and protected route access
Study Material APIs	Upload, retrieve, update and manage study materials
AI Feature APIs	AI summary, flashcard, quiz and study plan generation
Administrative APIs	Admin-only endpoints for managing users and monitoring system activity

6. Running the Backend

```
npm install
npm start
```

After successful execution the server connects to MongoDB and starts listening for incoming API requests. MongoDB must be running and the .env values configured first.